Comparing ReliableAutoML with Other AutoML Frameworks by Method Family: Protocol and Output Analysis

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—We specify a protocol for comparing ReliableAutoML with other open-source AutoML frameworks chosen by method family: stacking/meta-ensemble systems (AutoGluon-Tabular, H2O AutoML, auto-sklearn), a cost-frugal search system (FLAML), and an evolutionary pipeline-search system (TPOT). The comparison is of delivered *outputs*: the selected model, held-out balanced accuracy, accuracy and log-loss, fitting time under a stated budget, and the reports and artefacts each framework returns. All frameworks are fitted on identical development rows and scored once on identical untouched test rows for four datasets and three seeds. In this version the ReliableAutoML and plain-logistic-regression rows contain numbers measured on the authors' machine; the competitor cells are deliberately empty (marked *TBF*, "to be filled from the Colab run") because the competitors are run only in the accompanying Colab notebook. No competitor result is reported or implied here.**

***Index Terms*—automated machine learning, benchmarking, stacking, model selection, tabular classification**

I. Purpose and scope

ReliableAutoML (described in the companion paper) is designed around leakage control and reliability reporting: it validates the table, cleans and encodes inside folds, tunes candidates with nested cross-validation, and returns a `ReliablePipeline` whose reports (cleaning, partition, selection, reliability) accompany predictions. Other frameworks are designed primarily for predictive accuracy under a time budget. A fair comparison must therefore state what is compared. We compare (i) predictive outputs on untouched data, (ii) cost, and (iii) what each framework hands back to the user. We do *not* claim that any ranking on four small datasets generalises.

II. Choice of competitors by family

Frameworks are grouped by the mechanism that produces the final predictor, since mechanism determines both accuracy behaviour and the kind of output available.

(a) **Stacking / meta-ensemble.** The final predictor is a learned or selected combination of many base models. This is the family closest to ReliableAutoML, whose candidate pool contains cross-fitted stacking learners (`meta_pipeline`, `sc_meta`) and a performance-based selector, and which chooses among candidates by nested outer-fold score. Members: **AutoGluon-Tabular** [2] (multi-layer stack ensembling with k-fold bagging), **H2O AutoML** [5] (many base models followed by stacked ensembles, with cross-validation), and **auto-sklearn** [1] (Bayesian optimisation with meta-learning warm start and post-hoc ensemble selection). These are the primary comparators.

(b) **Cost-frugal search. FLAML** [3] allocates a time budget by estimated cost of learners and hyperparameter settings and typically returns a single tuned model. It is a contrast: it shows what a small, fast search delivers and how little output beyond the model it provides.

(c) **Evolutionary pipeline search. TPOT** [4] evolves scikit-learn pipelines by genetic programming. It is a contrast in search mechanism and in output: an exportable scikit-learn pipeline.

We include a plain logistic-regression pipeline (median/mode imputation, one-hot, standardisation, balanced class weights)

as a floor that is cheap and unselected; it shows when the search machinery adds nothing. Auto-sklearn runs only on Linux and, in current releases, on older Python versions; it is attempted in the Colab notebook and, if installation fails there, its cells stay empty and this is stated rather than substituted. Frameworks not chosen (e.g., commercial services, deep-learning-only systems) are excluded because they do not belong to these families or are not freely runnable by readers.

III. Protocol

For dataset D and seed s, a stratified split with test fraction 0.25 and `random_state`$= s$ yields development rows D_{dev} and test rows D_{test}. The test rows are used once, after fitting, for every framework; no framework or analyst chooses settings using them. Datasets: a synthetic mixed-type table (240 rows, two numeric and two categorical predictors with injected missing values, binary target), Wine (178 rows, 3 classes), Breast Cancer (569, 2) and Digits (1797 rows, 64 features, 10 classes), the last three from scikit-learn. Seeds: 42, 43, 44; the seed also seeds the framework when it accepts one and, for the synthetic data, the generator.

Metrics. With C classes, per-class recall $r_c = TP_c/(TP_c + FN_c)$, balanced accuracy $\text{BA} = C^{-1}\sum_c r_c$, accuracy, and log-loss $\text{LL} = -n^{-1}\sum_i \log \hat{p}_{i,y_i}$ with probabilities clipped to $[10^{-15}, 1]$ and renormalised. The same function computes these for all frameworks from their predicted labels and class probabilities. Reported values are mean±sample standard deviation over the three seeds ($n = 3$; the standard deviation is itself imprecise).

Time. Competitors receive a wall-clock budget $B = 60\,\text{s}$ per fit (notebook parameter `BUDGET`); TPOT receives it as `max_time_mins`. ReliableAutoML has no time-budget parameter: its cost is determined by the nested search ($K(G_m J + 1) + \ldots$ fits per candidate, see the companion paper), and we report its measured wall time instead. Budgets are therefore not matched: on Digits ReliableAutoML used about 290 s per fit, nearly five times B. Comparisons of accuracy at unequal cost must be read with this in mind; a second Colab pass with B set to the measured ReliableAutoML time per dataset is the natural control.

Internal objectives differ. ReliableAutoML optimises balanced accuracy; AutoGluon is configured with balanced accuracy; TPOT with balanced accuracy; auto-sklearn with balanced accuracy; H2O with mean per-class error (an equivalent for balanced error); FLAML uses its default (log-loss for classification). Differences in log-loss and balanced accuracy partly reflect these objectives.

Input handling. ReliableAutoML, AutoGluon, H2O, FLAML and auto-sklearn receive the raw development table including NaNs and string categoricals (auto-sklearn with categorical dtypes). TPOT requires numeric input, so a median/mode imputer and one-hot encoder fitted on D_{dev} precede it; this is a handicap or aid depending on the dataset and is noted in the results. The scripts are `experiments/automl_comparison.py` and `docs/colab/automl.ipynb`; the notebook embeds the same script.

Outputs recorded. Selected model identifier, leaderboard size, wall time, and for ReliableAutoML the presence and structure of cleaning, fit/partition, and reliability reports and the size of the saved artefact.

IV. Results measured locally

The following were measured on the authors' Windows machine (Python 3.15.0, NumPy 2.5.4, pandas 3.1.0, scikit-learn 1.9.1), by running only ReliableAutoML (defaults: `s_attp_clf`, light tuning, $K = 3$, $J = 2$, seven candidates including StatCore) and the logistic baseline. All 24 runs completed without error. Competitor rows are marked *TBF*: they will be filled from the Colab results file `automl_comparison_results.json`. Values are rounded to four decimals.

A. What the local numbers show

ReliableAutoML's balanced accuracy is above the logistic baseline's on Breast Cancer (by 0.003, within one baseline standard deviation) and Digits (by 0.008, slightly more than either method's standard deviation), and below it on the synthetic data (by 0.035) and Wine (by 0.009, far inside the run-to-run spread). Its log-loss was higher than the baseline's on all four datasets, with the largest gap on Wine, where one seed gave a high loss (the Wine standard deviation of 0.2098 reflects this). The selected candidate changes with seed (Table III), so the output "which model" is not stable on these small samples. ReliableAutoML cost between about 24 s and 290 s against a fraction of a second for the baseline. On this evidence, for these datasets the search buys no consistent accuracy gain over a regularised linear model, and its value must be argued from the reports it returns rather than from accuracy. With three seeds no significance test is meaningful, and we make none.

B. Artefacts returned by ReliableAutoML

For the first synthetic run the object returned: a leaderboard of 7 candidates; a cleaning report with sections `cleaning_intensity`, `data_integrity`, `ordinal_features`, `scaling_strategy`, `statistical_cleaning`; a fit report with `cleaning`, `leaderboard`, `partition_sizes`, `partitioner`, `selection`, `target_column`, `warnings` (one warning raised); and a `ReliabilityReport` from `predict_with_report` with fields including `covariate_shift_detected`, `drifted_features`, `mean_psi`, `similarity_score`, `confidence_intervals` and `evaluation`. The saved artefact was a single 30,459-byte file. These outputs describe data handling and drift; they do not by themselves improve held-out accuracy. Whether any competitor produces equivalents is the question Table II frames and the Colab notebook checks.

TABLE I

FRAMEWORKS BY FAMILY. ENTRIES REFLECT THE CITED PAPERS AND DOCUMENTED DEFAULTS; VERSIONS AND DEFAULTS CHANGE, AND "LEAKAGE CONTROLS" LISTS ONLY WHAT THE FRAMEWORK'S OWN DESIGN DOES, NOT A GUARANTEE. "NESTED" MEANS SELECTION IS EVALUATED ON FOLDS NOT USED FOR ITS OWN TUNING.

Framework	Family	Search strategy	Ensembling	Internal validation	Leakage controls
ReliableAutoML	(a) stacking + nested selection	Light grid or randomized search per fixed candidate pool (7 candidates by default)	Cross-fitted stacking candidates; single selected pipeline	Nested stratified CV: inner $J = 2$ search inside outer $K = 3$ folds	Fold-local cleaning/scaling/encoding; ATTP partition design exception documented; fresh external test advised
AutoGluon-Tabular	(a)	Fixed model portfolio, little hyperparameter search by default [2]	Multi-layer stacking, bagged folds	k-fold bagging; out-of-fold predictions feed upper layers	Out-of-fold stacking within the supplied training data; no nested estimate of the whole procedure
H2O AutoML	(a)	Default models plus random grid search [5]	Stacked ensembles (all models; best per family)	Cross-validation (nfolds) on training frame	Cross-validated metalearner on out-of-fold predictions; leaderboard metrics are optimistic for selection
auto-sklearn	(a)	SMAC Bayesian optimisation with meta-learning warm start [1]	Post-hoc ensemble selection over evaluated models	Holdout or cross-validation on training data	Validation predictions drive ensemble; no nested estimate
FLAML	(b) cost-frugal	Cost-aware search over learners/configurations [3]	Single best model by default (optional ensemble)	Holdout or CV chosen from data size	Internal validation on training data; no nested estimate
TPOT	(c) evolutionary	Genetic programming over scikit-learn pipelines [4]	None in the evolved pipeline unless stacking operators are evolved	Cross-validation of each pipeline (fitness)	Preprocessing inside evaluated pipelines; fitness is the CV score, optimistic for the winner

TABLE II

OUTPUT AND PRACTICALITY COMPARISON. ENTRIES ARE QUALITATIVE STATEMENTS FROM DESIGN AND DOCUMENTATION; *TBF* MARKS ITEMS THE COLAB RUN (OR THE NOTEBOOK'S PRINTED OUTPUTS) SHOULD CONFIRM FOR THE INSTALLED VERSIONS.

Framework	Time-budget parameter	Leaderboard of candidates	Cleaning / partition / reliability report	Interpretability outputs	Exportable pipeline	Main dependencies	Licence
ReliableAutoML	No (cost set by grid and folds)	Yes, with outer mean/std and fold scores	Cleaning, partition, selection reports and `ReliabilityReport` (drift, intervals) from `predict_with_report`	Explanation-consistency and reason audit	Pickle of `ReliablePipeline` (sklearn pipeline inside)	NumPy, pandas, scikit-learn, SciPy, ATTP	MIT
AutoGluon-Tabular	Yes	Yes	No reliability report; no cleaning report	Feature importance	Predictor directory (not a plain sklearn object)	PyTorch and gradient-boosting libraries	Apache-2.0
H2O AutoML	Yes	Yes	No	Variable importance, SHAP via H2O explain	Native model/MOJO export	Java runtime	Apache-2.0
auto-sklearn	Yes	Yes (evaluated runs)	No	Limited (ensemble weights)	Fitted object (pickle)	scikit-learn, SMAC; Linux only	BSD-3
FLAML	Yes	Best model and search history	No	Feature importances of best learner	Estimator object (pickle)	Light; learner libraries optional	MIT
TPOT	Yes (generations/time)	Pareto set of pipelines	No	None built in	Fitted scikit-learn pipeline; code export in classic versions	scikit-learn	LGPL-3.0

V. INTERPRETING THE COMPETITOR RESULTS WHEN THEY ARRIVE

Pre-specified expectations, to be confirmed or refuted, not asserted: stacking frameworks given a 60 s budget may exceed ReliableAutoML on the larger Digits data and may lose or tie on small datasets where all models are near ceiling; FLAML may match accuracy at lower cost but returns fewer artefacts; TPOT's result depends on its short budget and on external preprocessing. Any claim that ReliableAutoML is better than a competitor on accuracy requires the filled table and a stated uncertainty. Where a competitor is higher, we will report that plainly and attribute ReliableAutoML's niche to leakage control, nested selection and reporting.

VI. THREATS TO VALIDITY AND LIMITATIONS

(1) Four small datasets and three seeds; no significance testing. (2) Unmatched budgets and objectives. (3) ReliableAutoML's partition step reduces the rows used for its final fit relative to some frameworks (it trains on the ATTP training partition of D_{dev}, e.g. 144 of 180 rows on the synthetic data), whereas competitors typically use all of D_{dev}; this disadvantages ReliableAutoML in accuracy comparisons. (4) The logistic baseline uses all of D_{dev}. (5) Default settings of competitors change across versions; the Colab run records installed versions. (6) Competitor runs on Colab use different hardware from the local ReliableAutoML runs, so time comparisons across the two environments are indicative only; for a clean time comparison, run ReliableAutoML in the same notebook

TABLE III

HELD-OUT RESULTS, MEAN±STD OVER SEEDS 42–44. n_{dev}/n_{test} AND FEATURE/CLASS COUNTS ARE FOR ONE SPLIT. TIME IS WALL-CLOCK SECONDS FOR THE FULL FIT. *TBF* = TO BE FILLED FROM THE COLAB RUN.

Dataset (n_{dev}/n_{test})	Framework	Balanced acc.	Accuracy	Log-loss	Time (s)	Budget (s)	Selected model
Synthetic mixed (180/60)	ReliableAutoML	0.7932±0.0211	0.7944±0.0192	0.4030±0.0117	23.8±7.6	none	`meta_pipeline`, `sc_logistic`, `met`
	Logistic baseline	0.8281±0.0295	0.8278±0.0255	0.3703±0.0037	0.0	–	–
	AutoGluon-Tabular	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	H2O AutoML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	auto-sklearn	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	FLAML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	TPOT	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
Wine (133/45)	ReliableAutoML	0.9568±0.0466	0.9481±0.0559	0.2281±0.2098	40.8±7.1	none	`meta_pipeline` (2), `perf_meta`
	Logistic baseline	0.9660±0.0298	0.9630±0.0339	0.0883±0.0344	0.0	–	–
	AutoGluon-Tabular	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	H2O AutoML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	auto-sklearn	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	FLAML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	TPOT	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
Breast Cancer (426/143)	ReliableAutoML	0.9652±0.0037	0.9627±0.0107	0.1412±0.0292	73.2±9.1	none	`sc_mlp` (2), `meta_pipeli`
	Logistic baseline	0.9621±0.0144	0.9604±0.0081	0.0921±0.0349	0.0	–	–
	AutoGluon-Tabular	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	H2O AutoML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	auto-sklearn	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	FLAML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	TPOT	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
Digits (1347/450)	ReliableAutoML	0.9799±0.0042	0.9800±0.0044	0.1260±0.0586	290.0±8.1	none	`sc_meta` (2), `sc_knn`
	Logistic baseline	0.9717±0.0067	0.9719±0.0068	0.1084±0.0087	0.1	–	–
	AutoGluon-Tabular	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	H2O AutoML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	auto-sklearn	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	FLAML	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*
	TPOT	*TBF*	*TBF*	*TBF*	*TBF*	60	*TBF*

(it is in the framework list). (7) The Digits and Wine/Breast Cancer data are standard benchmarks and unrepresentative of messy industrial tables. (8) Selection among frameworks after viewing test scores would invalidate them; the framework list and metrics were fixed in advance. (9) Table II states design-level facts, not measured behaviour.

VII. REPRODUCTION

Local (ReliableAutoML only): `python experiments/automl_comparison.py -frameworks reliableml sklearn_logistic -output experiments/automl-comparison-results.json`. Colab: open `docs/colab/automl.ipynb`, set the parameter cell, run all, download `automl_comparison_results.json` and `automl_comparison_table.csv`, and paste the values into Table III.

REFERENCES

[1] M. Feurer *et al.*, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015.

[2] N. Erickson *et al.*, "AutoGluon-Tabular: Robust and accurate AutoML for structured data," arXiv:2003.06505, 2020.

[3] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," in *Proc. Machine Learning and Systems (MLSys)*, 2021.

[4] R. S. Olson, N. Bartley, R. J. Urbanowicz, and J. H. Moore, "Evaluation of a tree-based pipeline optimization tool for automating data science," in *Proc. GECCO*, 2016.

[5] E. LeDell and S. Poirier, "H2O AutoML: Scalable automatic machine learning," 7th ICML Workshop on Automated Machine Learning (AutoML), 2020.

[6] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, article 91, 2006.

[7] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *JMLR*, vol. 11, pp. 2079–2107, 2010.

[8] J. Monchwe *et al.*, ReliableML-AutoML, software repository, 2026. https://github.com/Jabualani/ML-Reliabilty.